FALCON: Fast and Scalable Out-of-Core Training of Graph Neural Networks

석사과정 박세준

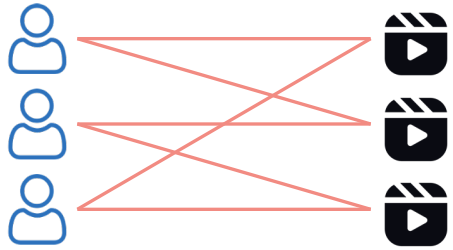
Contents

- **Graph Data**
- **Graph Neural Network**
- **Sampling Based Graph Learning**
- **In Memory Mini-batch Learning**
  - PyG
  - DGL
- **Out of Core Training**
  - Ginex
  - MariusGNN
- **Remaining Challenges**
- **FALCON**
  - Overview
  - Offline Clustering
  - Structure Preserving Graph Sampling
  - Community Aware Neighbor Sampling
  - Out of Core Training
- **Experiments**

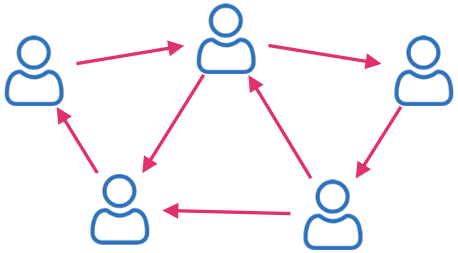
What is Graph Data?

- Users, videos, and products naturally form **graphs**, and one trained GNN serves many tasks.

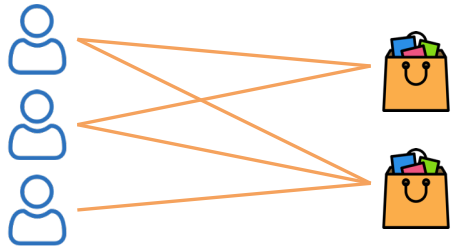
watch history



who follows whom



purchase history



Training

learns node embeddings

Recommendation

Next video to watch, accounts to follow, products to buy

Classification

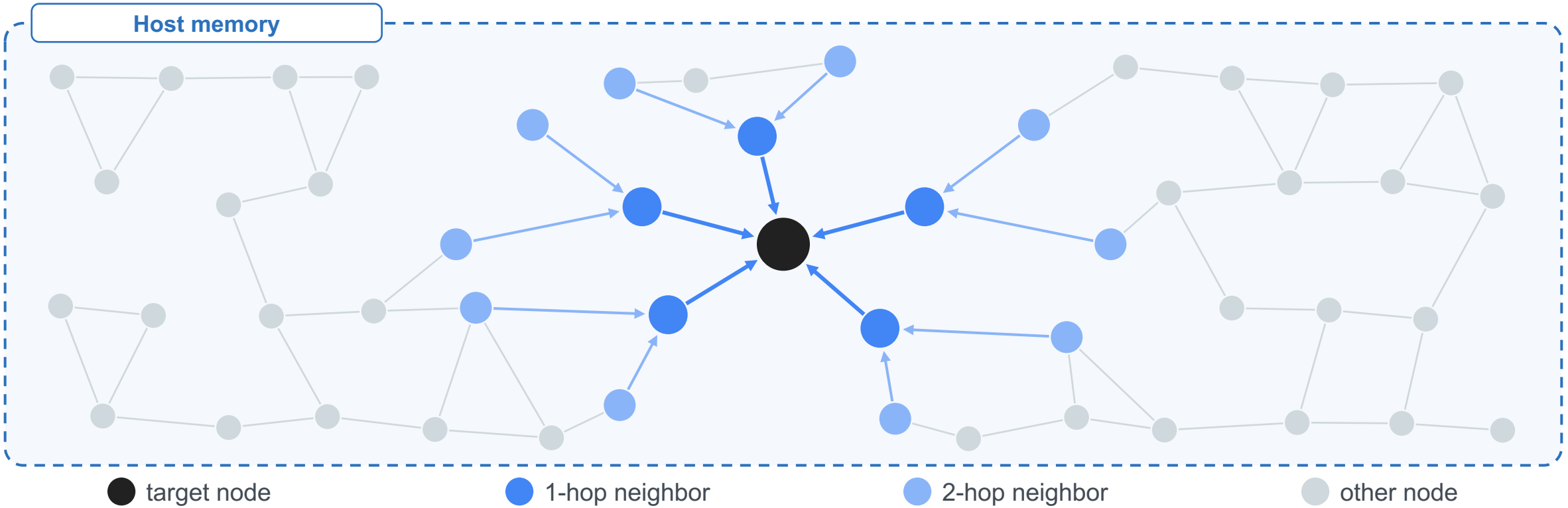
Label each node
e.g., fake or spam accounts, product categories

Grouping

Cluster similar nodes
e.g., friend communities, customer segments

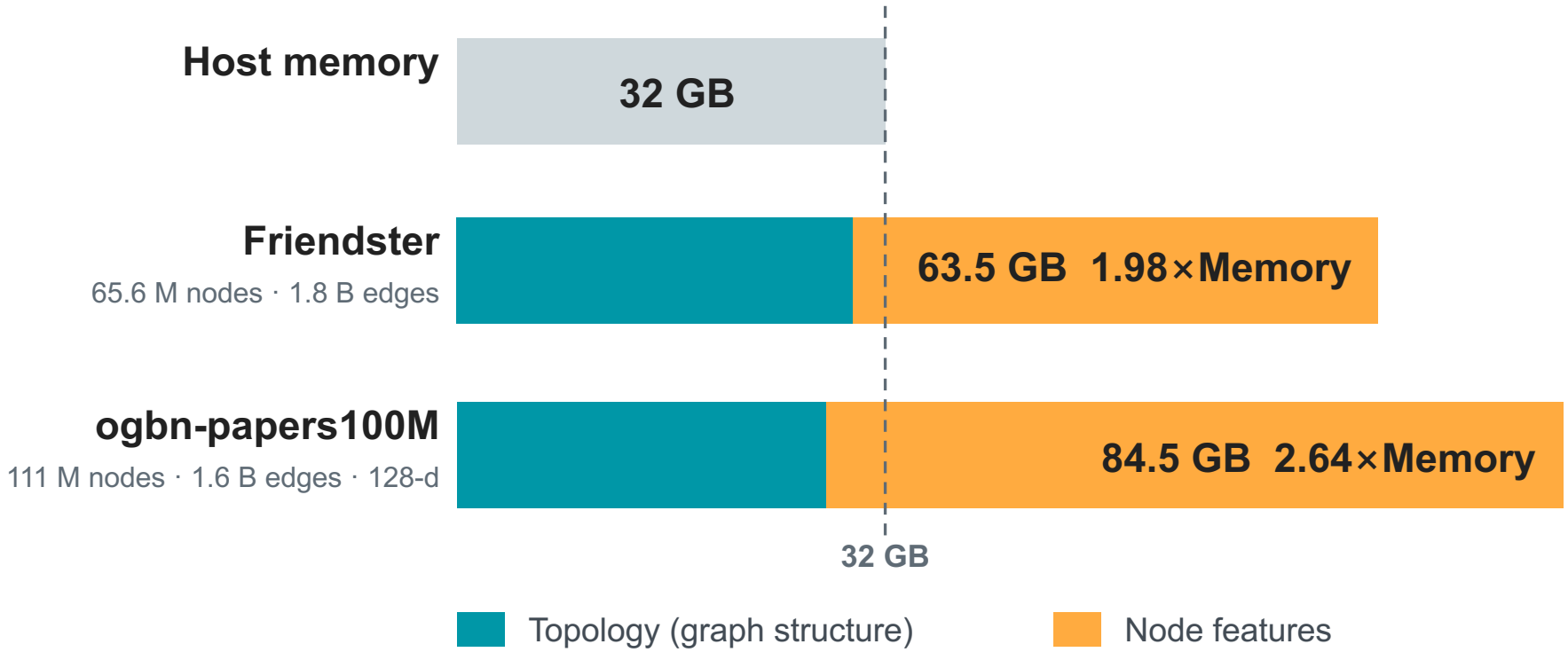
What Is a GNN, and How Is It Trained?

- A GNN learns each node's embedding by **aggregating information from its neighbors**, hop by hop.
- Sampled neighbors can be any node, so **every node's features must be in memory**.



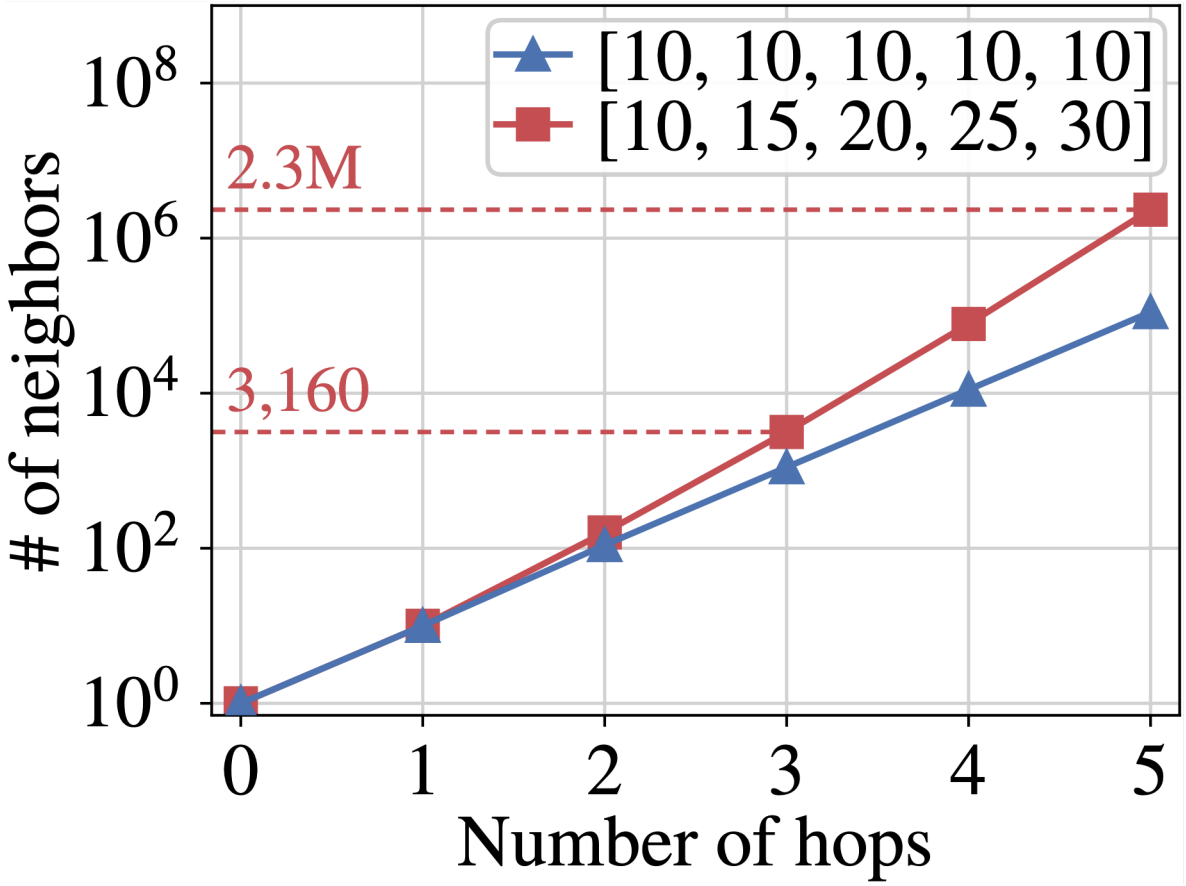
What about Large Scale Graph Data?

- Datasets nowadays are far larger than the memory of an ordinary machine.



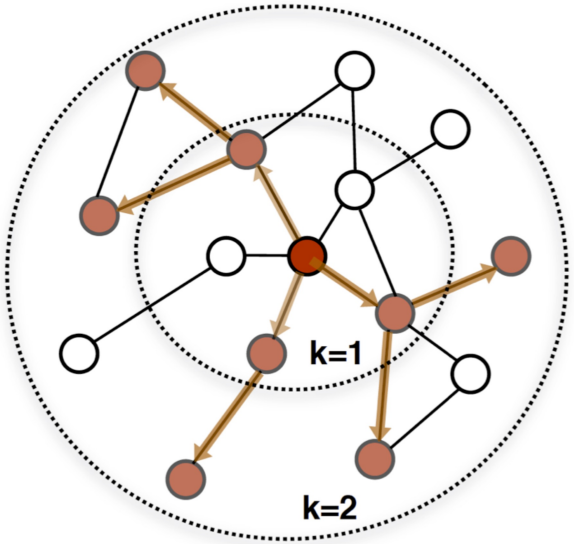
What about Large Scale Graph Data?

- Datasets nowadays are far larger than the memory of an ordinary machine.

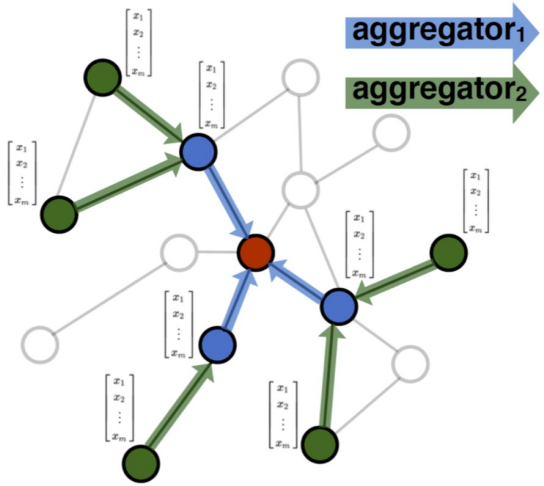


Sampling Based Graph Learning

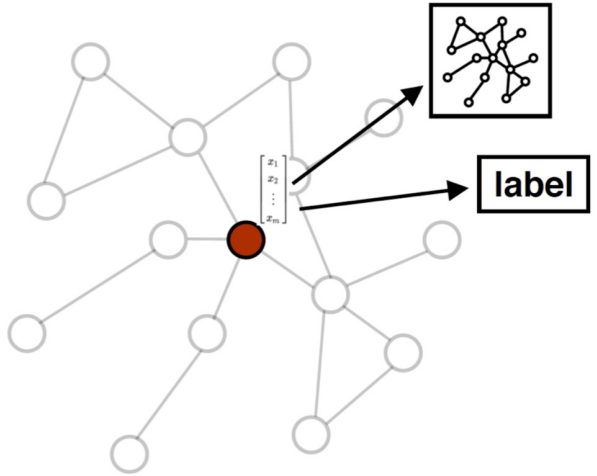
- Sample a fixed number of neighbors per hop for each mini-batch
- Aggregate sampled neighbor features to learn node representations
- Use the learned representations for prediction and model training



1. Sample neighborhood



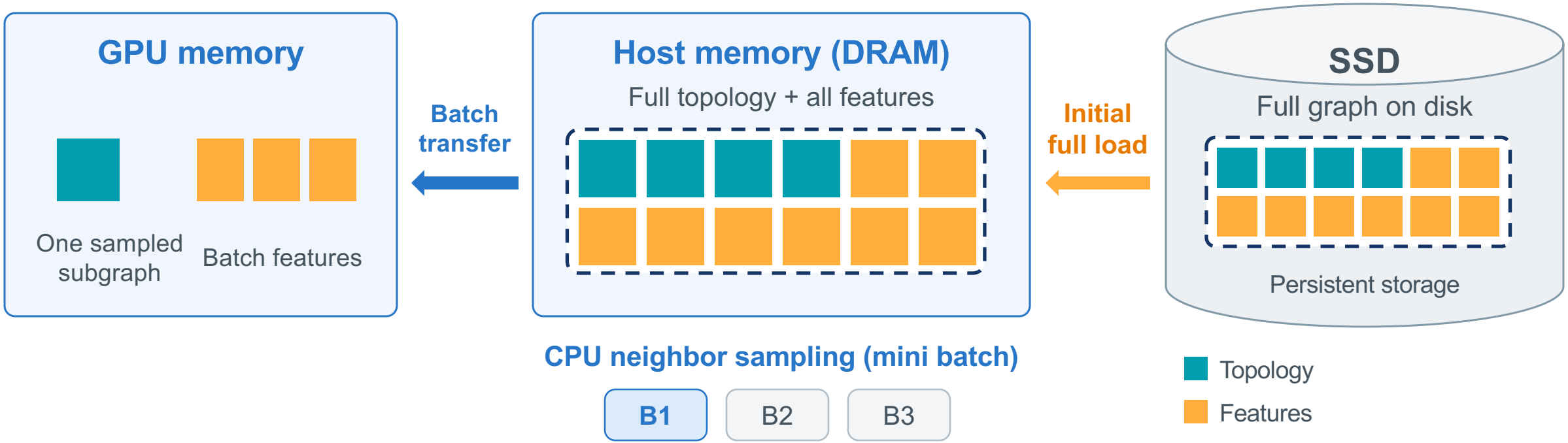
2. Aggregate feature information from neighbors



3. Predict graph context and label using aggregated information

PyG: In-memory mini-batch training

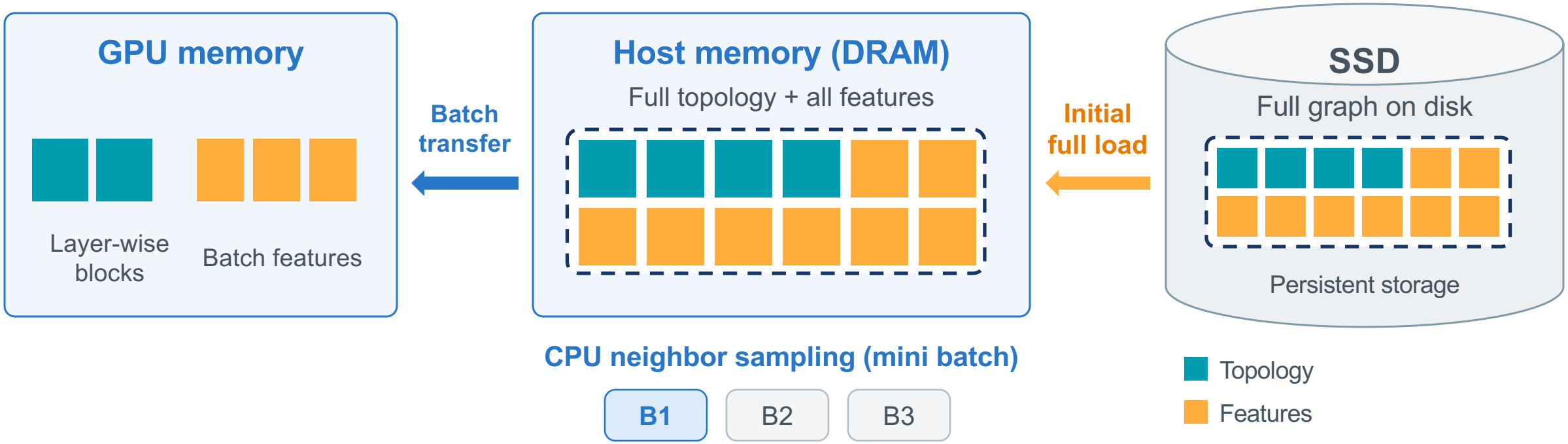
- Keep the full topology and features in host RAM
- Transfer only the current mini-batch



Sampled subgraph → GNN layer 1 → GNN layer 2 → Batch loss

DGL: In-memory mini-batch training

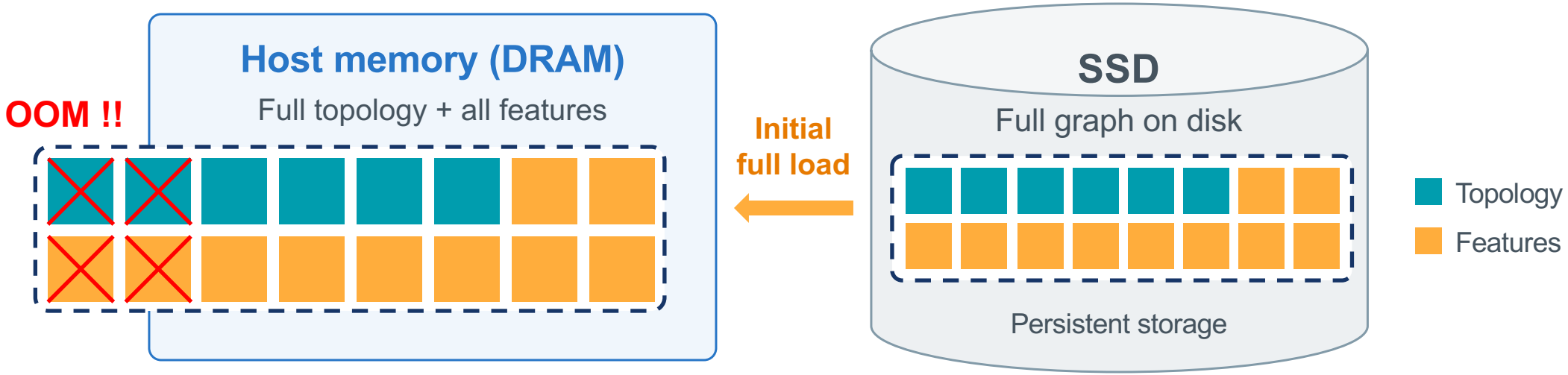
- Keep the full topology and features in host RAM
- Transfer only the current mini-batch



Block 1 + GNN layer 1 → Updated node features → Block 2 + GNN layer 2 → Batch loss

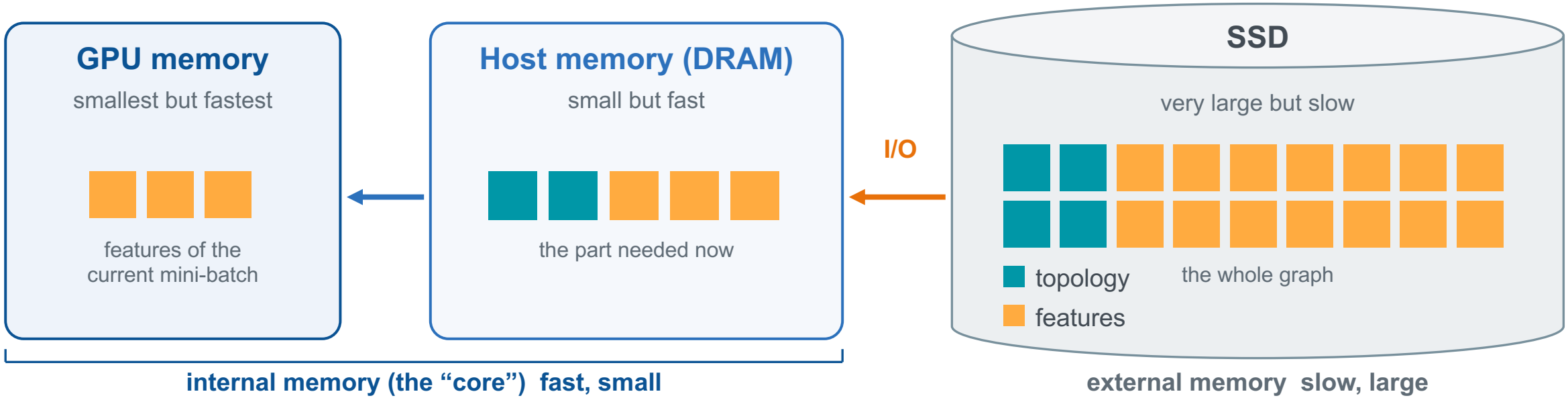
In Memory Mini-batch Training's Problem

- The full graph structure and features must fit in host memory
- If sampling and data transfer cannot keep up with GPU computation, the GPU waits for the next mini-batch.



Out-of-Core Training

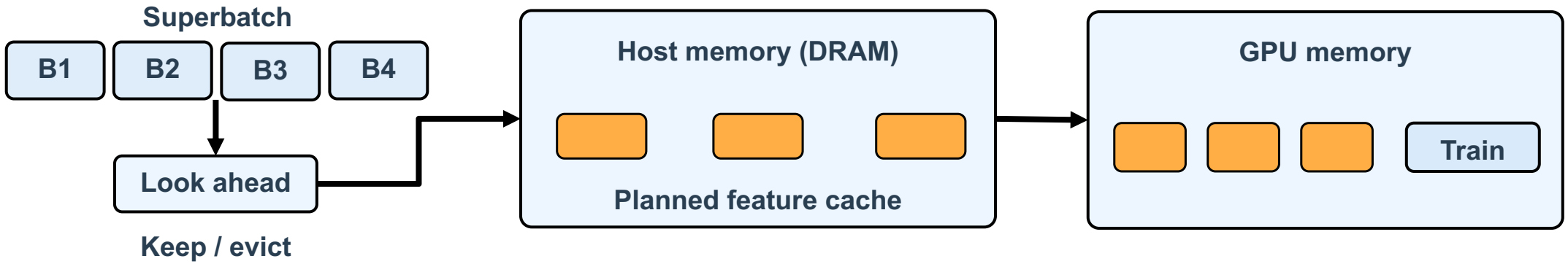
- Keep the graph on SSD and load only what each step needs
- So I/O becomes part of training



Out-of-core: **data that exceeds main memory lives on disk and is explicitly moved in as needed.**

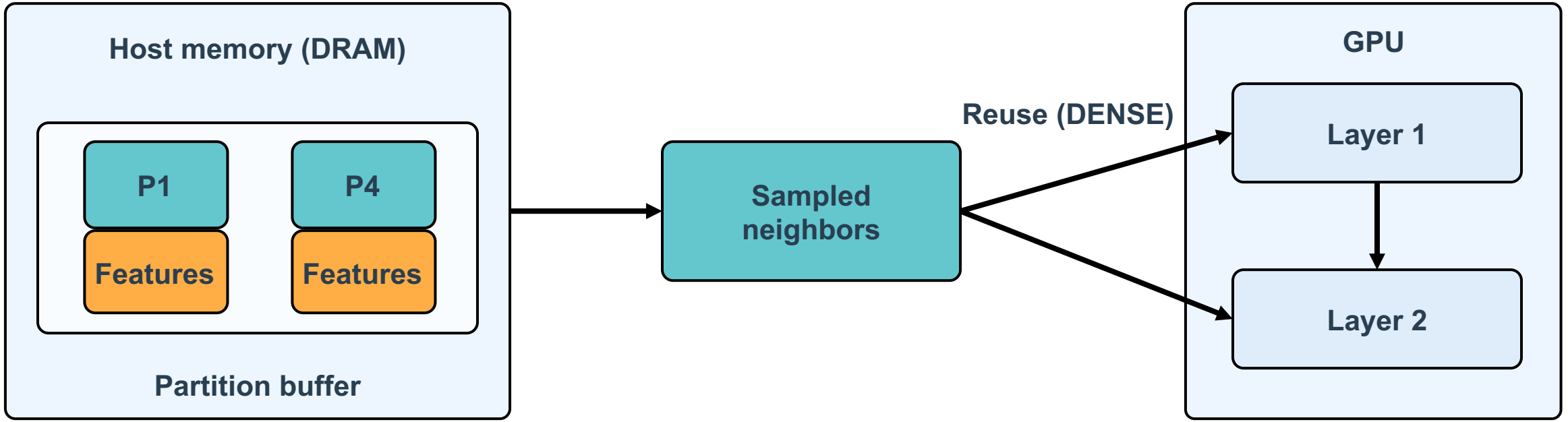
Out of Core Training-Ginex

- Sample a superbatch to identify future feature accesses
- Plan DRAM feature-cache replacement using Belady's policy
- Gather features and transfer each mini-batch to the GPU



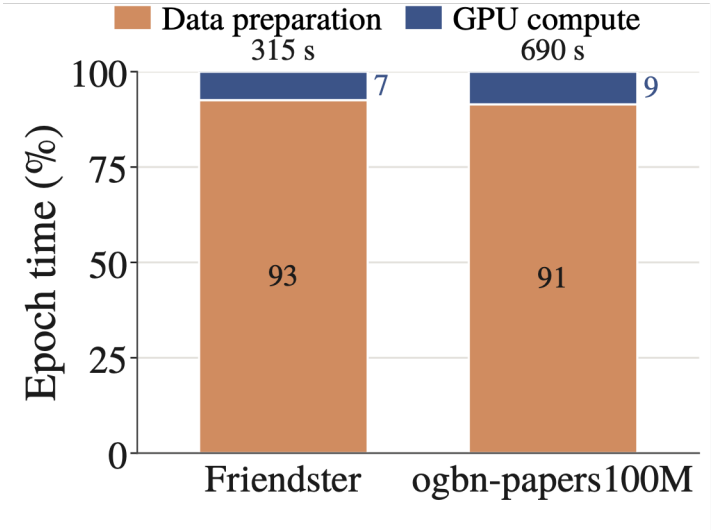
Out of Core Training-MariusGNN

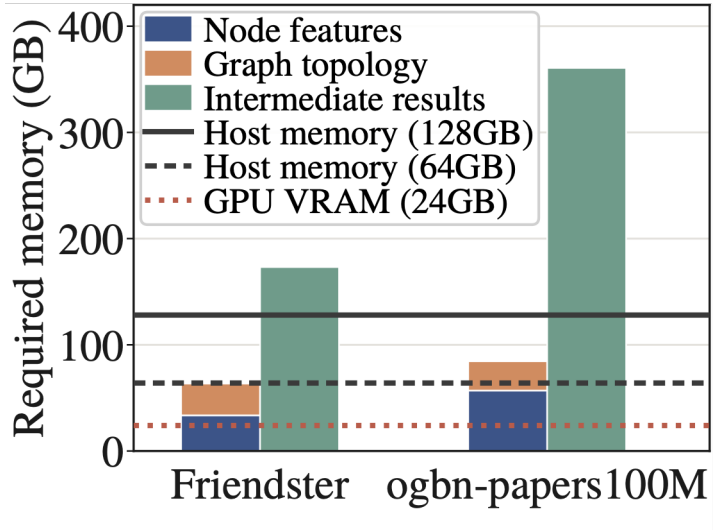
- Load a subset of graph partitions into DRAM
- Reuse sampled neighbors across layers with DENSE
- Train on the GPU using mini-batches from the resident graph

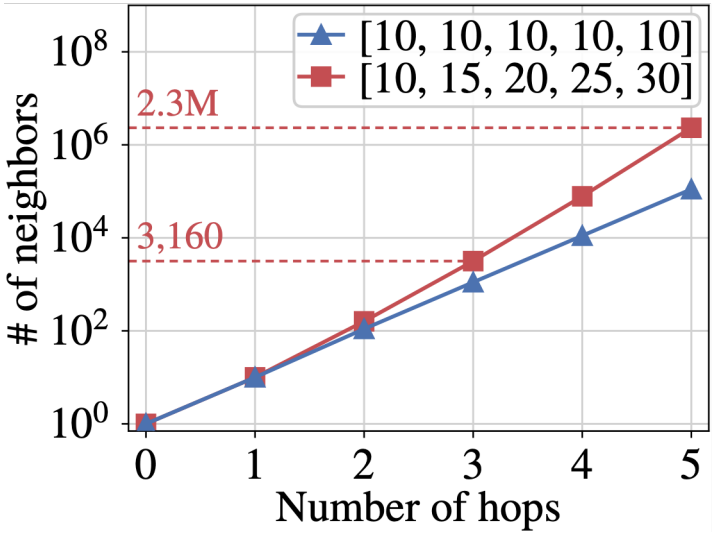


Remain Two Major Challenges

- **C1 Large Underlying Graph Size**
  - Sampling based GNN training still requires access to the entire graph
  - Large graphs incur substantial memory and I/O overhead
- **C2 Large Search Space for Neighbors**
  - Multi-hop sampling rapidly increases sampling and computation costs
  - Restricting the sampling space may degrade model accuracy

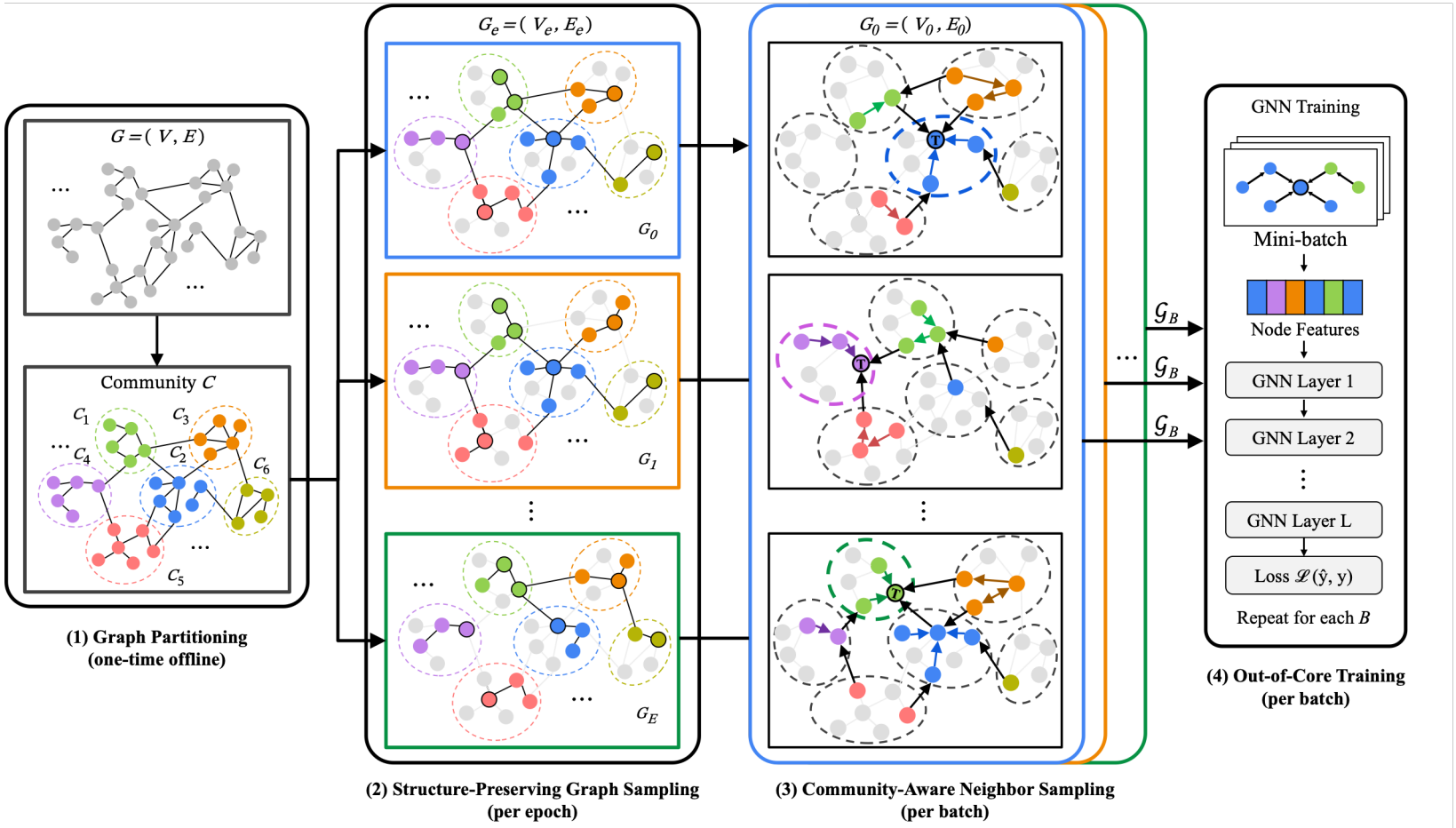






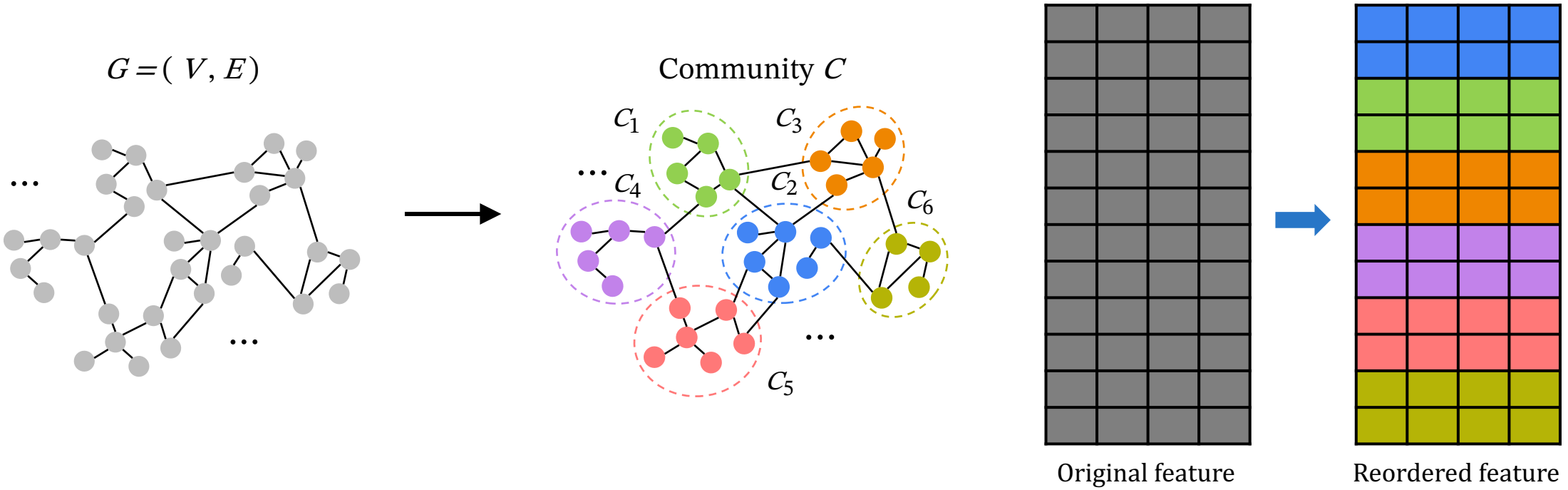
FALCON: Fast and Scalable Out-of-Core Training of Graph Neural Networks

- Achieves up to 11× speedup over the best competitor while maintaining comparable predictive accuracy
- Validates the effectiveness of each proposed strategy through comprehensive analyses on three real-world graphs



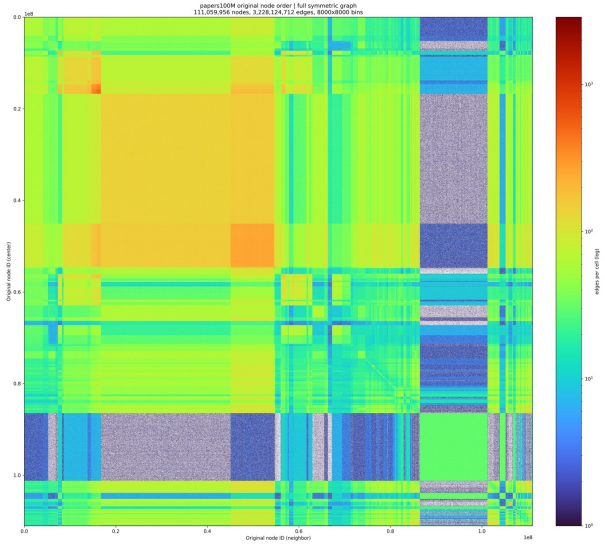
One Time Offline Graph Partitioing

- Can use any other clustering method like Modularity
- One Time Offline Graph Partitioing with METIS
- Original node features are reordered according to the graph partitioning results

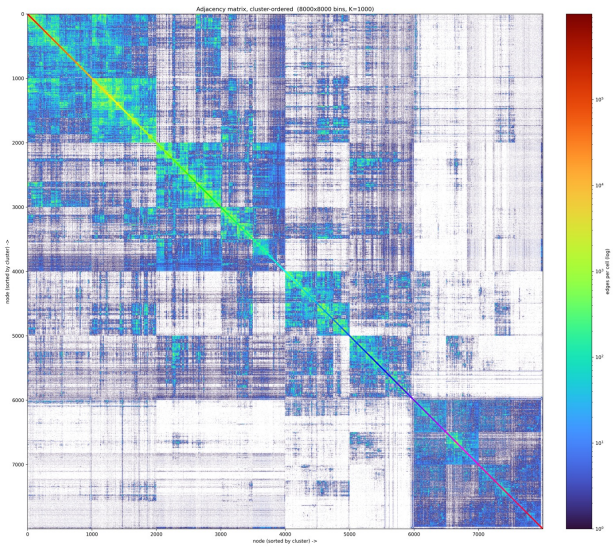


One Time Offline Graph Partitioing

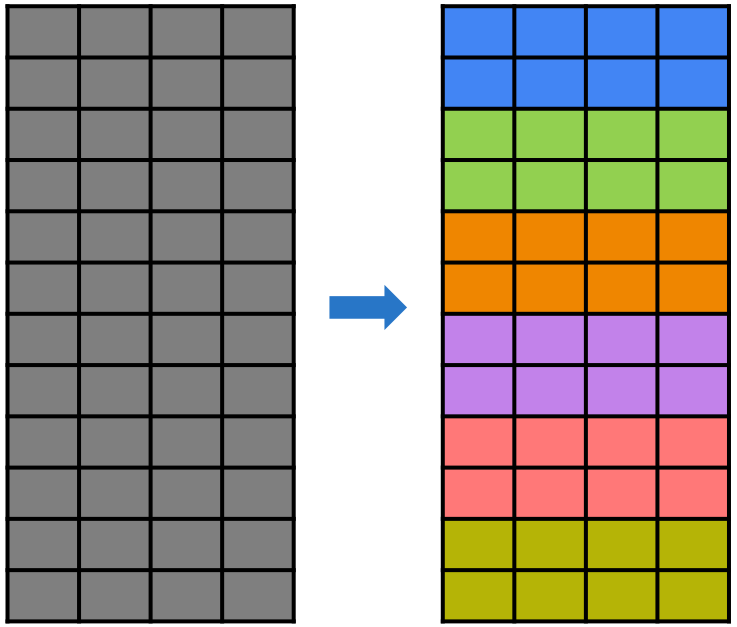
- Can use any other clustering method like Modularity
- One Time Offline Graph Partitioing with METIS
- Original node features are reordered according to the graph partitioning results



Original Data



Graph Partitioned Data

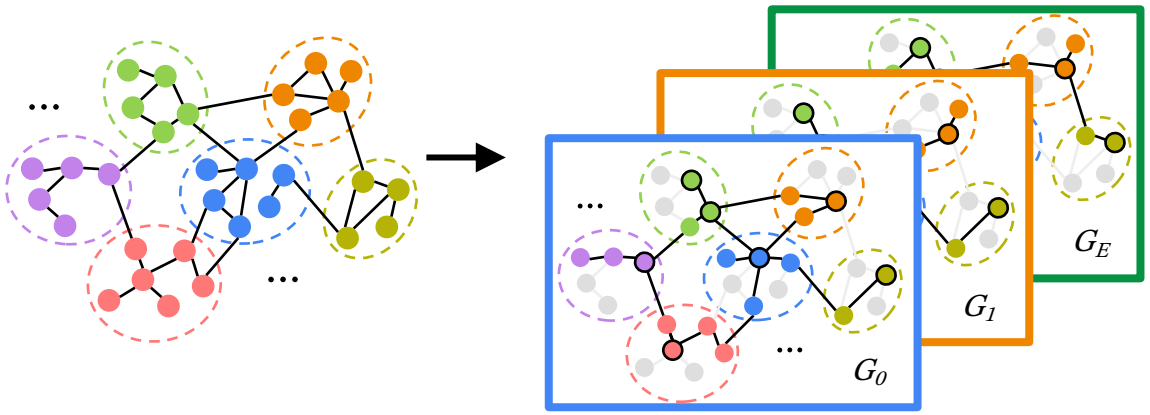


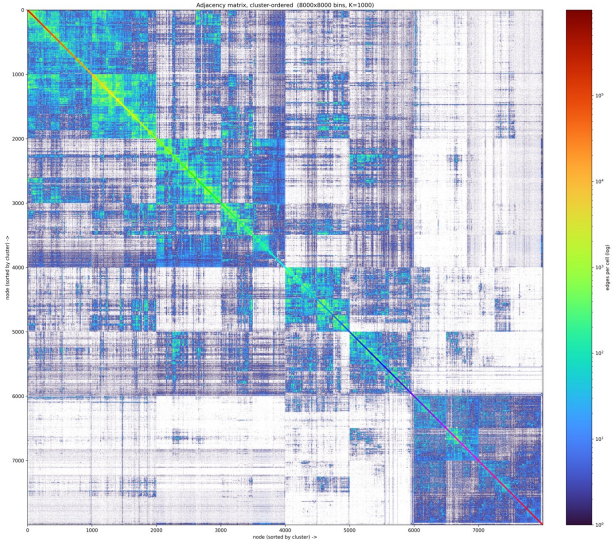
Original feature

Reordered feature

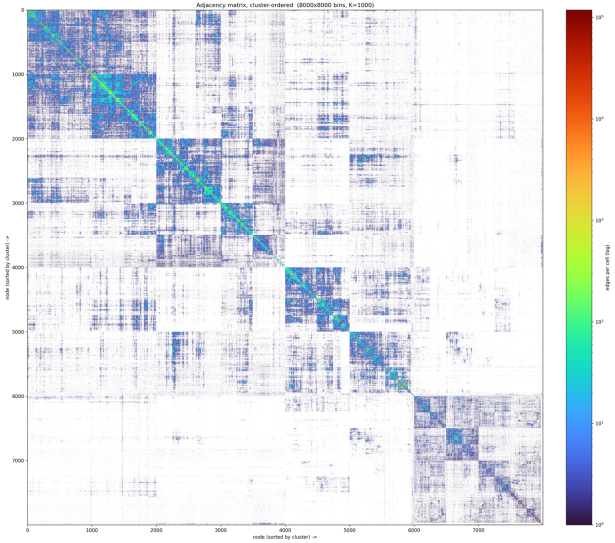
Structure Preserving Graph Sampling

- Use P_g to reduce non-training nodes while preserving all training nodes
- Sample within each community to preserve community structure
- Resample the graph at the beginning of each epoch





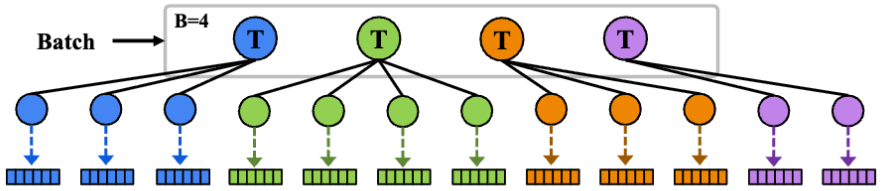
Before Graph Sampling Data



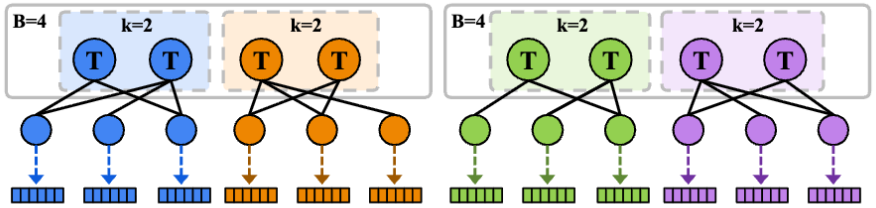
After Graph Sampling Data

Make Blocks and Shuffle

- We make blocks which are consist of only train seeds
- Have 4 stages to make train seed order
  - Grouping by cluster
  - Shuffled within cluster
  - Blocking the train seeds with block size
  - Shuffled that blocks



(a) Sampling without community-aware node blocking

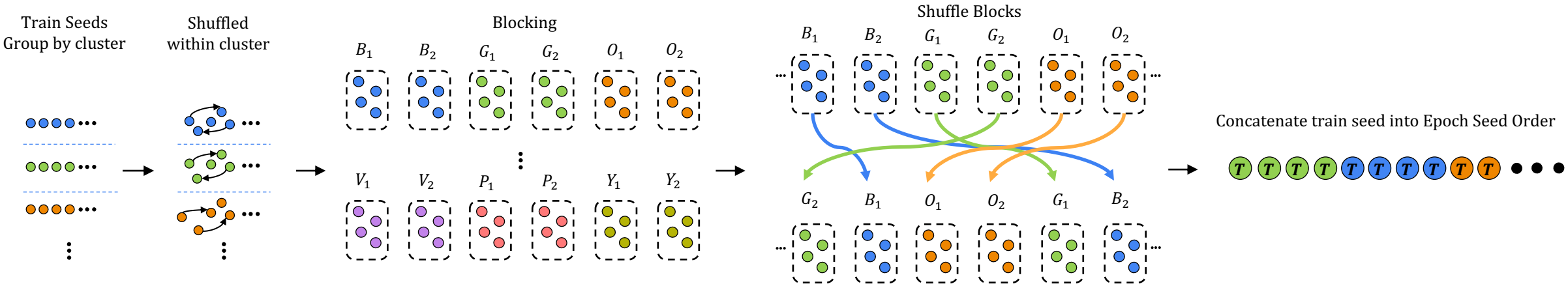


(b) Sampling with community-aware node blocking

Method	Unique Nodes
Global shuffle	$341{,}751 \pm 132$
Block only	$104{,}057 \pm 28$
FALCON	$231{,}046 \pm 191$

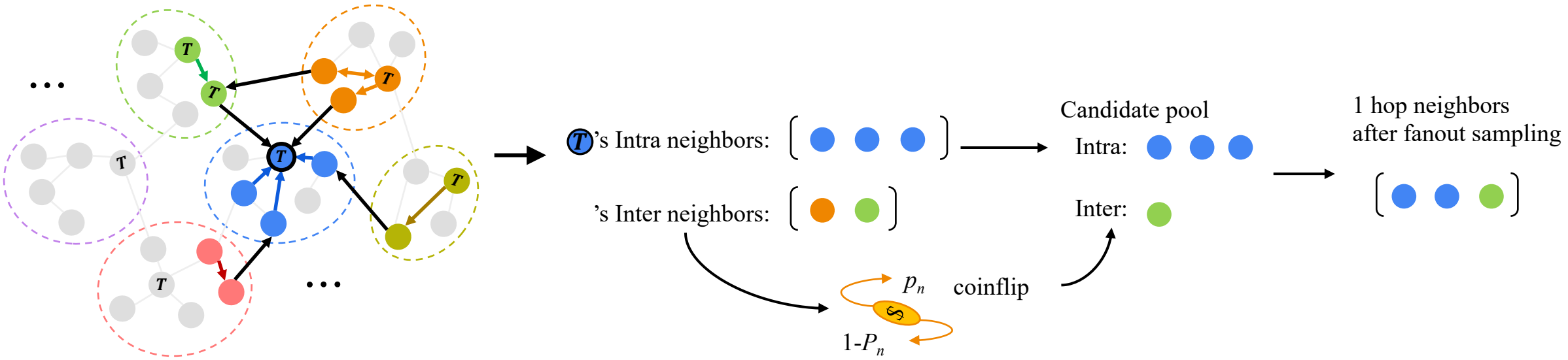
Make Blocks and Shuffle

- We make blocks which are consist of only train seeds
- Have 4 stages to make train seed order
  - Grouping by cluster
  - Shuffled within cluster
  - Blocking the train seeds with block size
  - Shuffled that blocks



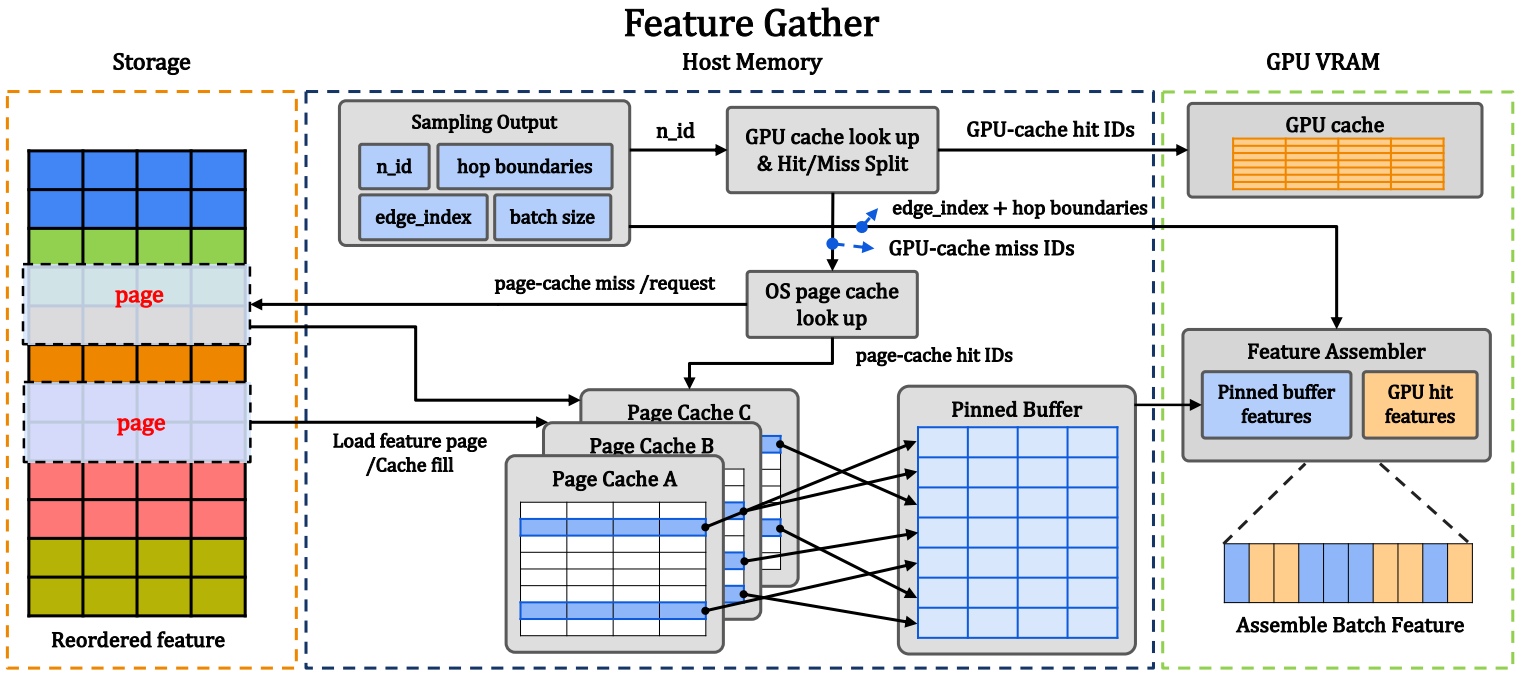
Community Aware Neighbor Sampling

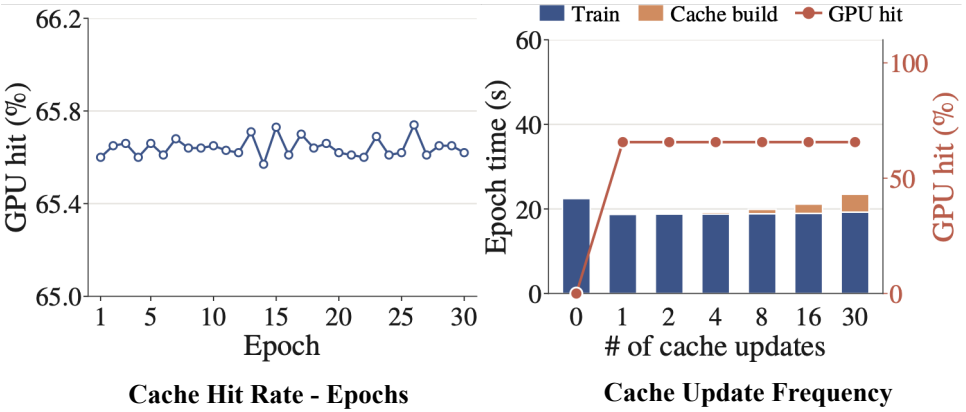
- Keep all intra-community neighbors and include inter-community neighbors with probability P_n in the candidate pool
- Uniformly sample the specified fanout from the candidate pool, repeating the process at each hop
- Reduce the neighbor search space and encourage shared neighbors to improve data locality



Out of Core Training

- Cache frequent node features using epoch-0 access counts and keep the GPU cache fixed
- Handle GPU cache misses via the OS page cache and SSD, gathering features into a pinned buffer
- Transfer gathered features to the GPU and combine them with cached features for GNN training





Cache Hit Rate - Epochs

Cache Update Frequency

Experiments

- **Our environments**
  - GPU: NVIDIA RTX 5080 (16 GB VRAM)
  - CPU: AMD Ryzen 9 9900X (12 Physical Cores)
  - Host Memory: 128 GB
  - Storage: PCIe Gen 4.0 NVMe SSD 2 TB

- **Used datasets**

Dataset	$\|V\|$	$\|E\|$	Topology	Feat.	Avg. Deg.
ogbn-products	2.45M	61.8M	1.0G	1.0G	50.5
ogbn-papers100M	111M	1.6B	27.6G	56.9G	29.1
Friendster	65.6M	1.8B	29.9G	33.6G	55.1

Dataset	$\|V_{\text{train}}\|$	$\|V_{\text{valid}}\|$	$\|V_{\text{test}}\|$
ogbn-products	197K (8.03%)	39.3K (1.61%)	2.21M (90.37%)
ogbn-papers100M	1.21M (1.09%)	125K (0.11%)	214K (0.19%)
Friendster	656K (1.00%)	–	–

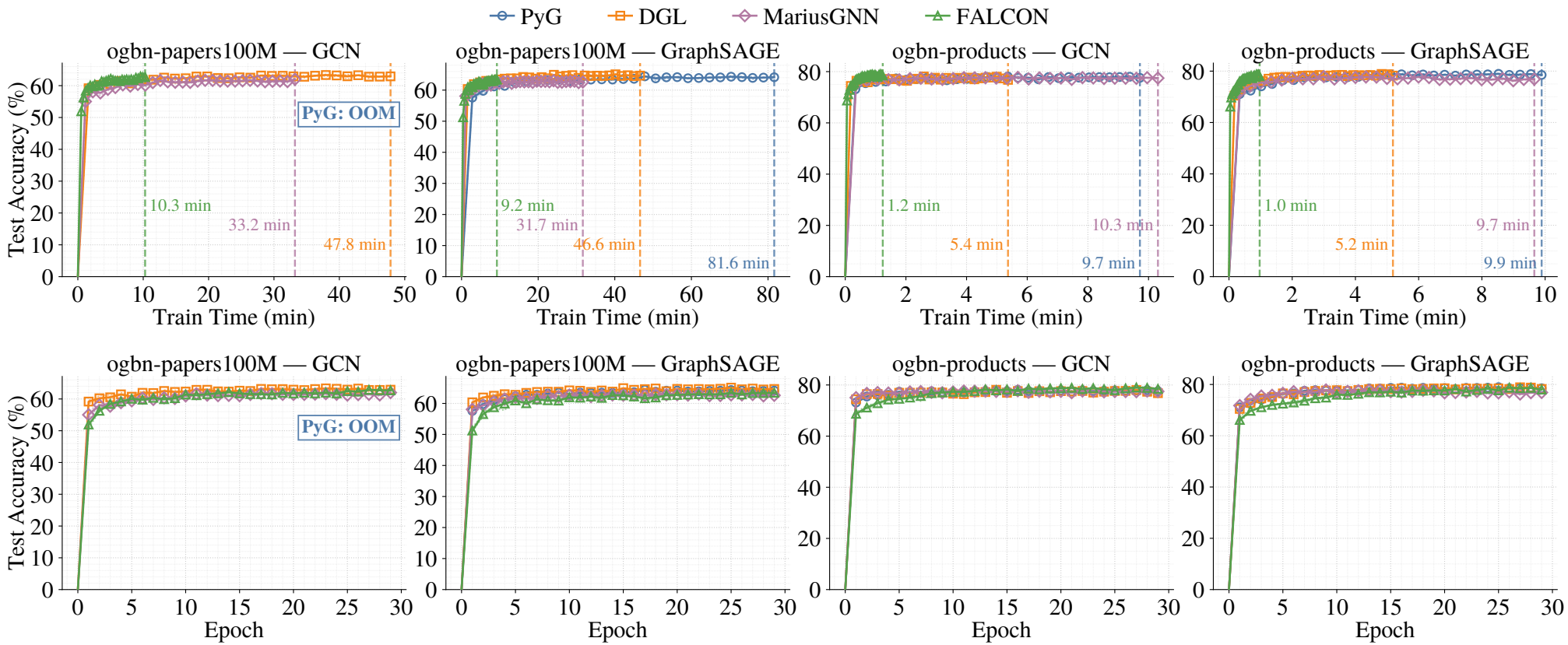
Experiments

- **EQ 1 Overall training efficiency**
  - FALCON achieves the shortest training time in most settings
  - PyG and DGL encounter OOM on large graphs with limited host memory
  - FALCON achieves up to 149.2× speedup over Ginex and 11.0× over MariusGNN, with Friendster at 64 GB as the only exception.

Host Mem.	Model	Method	ogbn-products		ogbn-papers100M		Friendster	
			Time	Memory	Time	Memory	Time	Memory
64GB	GCN	PyG	20.79	11.9	–	OOM	–	OOM
		DGL	12.17	4.5	–	OOM	–	OOM
		Ginex	257.21	3.1	1139.89	21.1	673.74	10.0
		MariusGNN	22.44	3.3	75.21	12.3	**28.30**	7.0
		FALCON	**2.61**	2.9	**53.98**	12.1	33.04	9.3
	GraphSAGE	PyG	21.12	11.3	–	OOM	–	OOM
		DGL	11.90	5.5	–	OOM	–	OOM
		Ginex	260.13	3.0	1153.31	18.7	674.06	10.9
		MariusGNN	21.06	3.6	71.93	11.9	**27.42**	7.1
		FALCON	**2.63**	2.7	**54.54**	13.1	31.19	10.0
128GB	GCN	PyG	20.10	12.1	–	OOM	82.93	98.7
		DGL	11.10	4.8	99.00	83.2	52.34	63.7
		Ginex	247.62	3.1	1064.77	21.7	614.83	11.7
		MariusGNN	21.32	3.3	68.69	12.6	25.44	7.1
		FALCON	**2.32**	2.9	**20.02**	12.4	**15.35**	9.3
	GraphSAGE	PyG	20.46	10.9	168.89	113.4	85.98	99.6
		DGL	10.73	4.7	96.48	83.1	51.28	65.3
		Ginex	271.62	5.0	1149.20	21.0	608.07	9.9
		MariusGNN	19.98	3.3	65.44	11.6	24.21	7.7
		FALCON	**1.82**	2.8	**18.14**	12.3	**15.15**	8.8

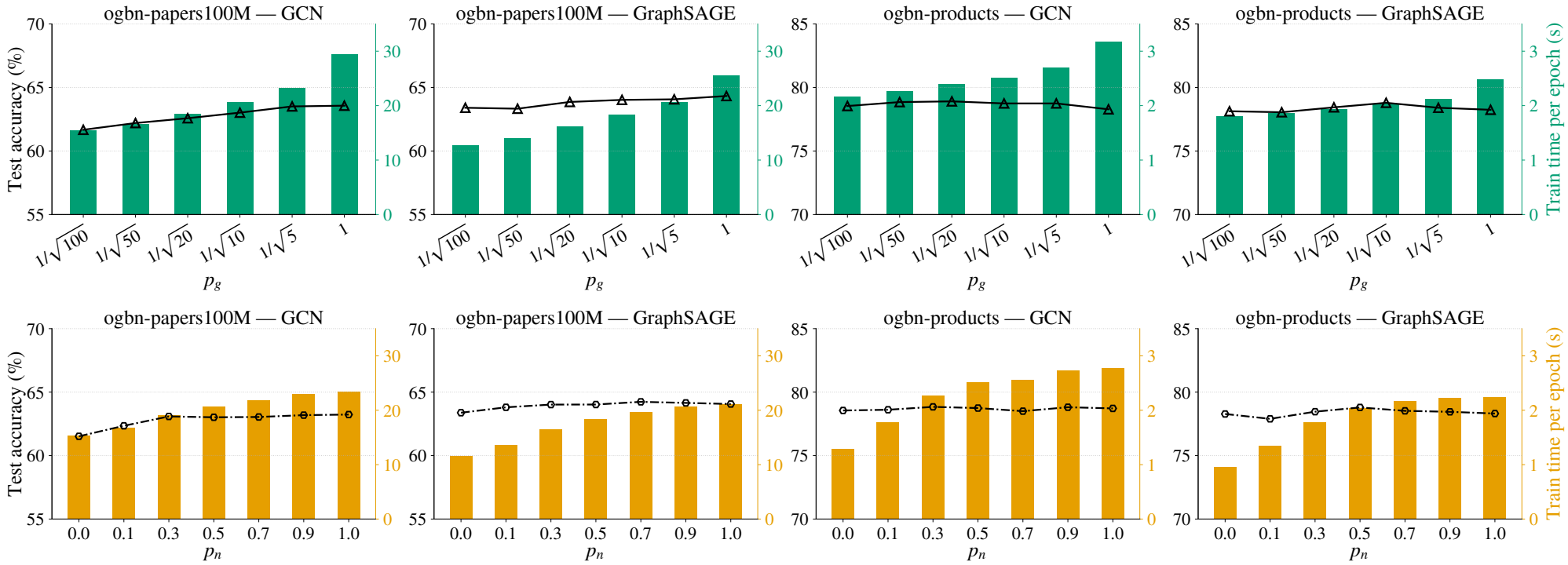
Experiments

- **EQ 2 Convergence rate analysis**
  - FALCON achieves accuracy competitive with PyG and DGL while consistently outperforming MariusGNN in converged accuracy.
  - On ogbn-papers100M, FALCON completes 30 epochs in 10.3 minutes for GCN and 9.2 minutes for GraphSAGE
  - Preserve structural information while reducing graph and neighbor search space for faster convergence with minimal accuracy loss



Experiments

- **EQ 3 Sampling strategy**
  - Structure-preserving graph sampling reduces training time while comparable accuracy
  - Community-aware neighbor sampling lowers data preparation costs with comparable accuracy
  - The default (p_n = 0.5) balances training efficiency and predictive accuracy



Experiments

- **EQ 4 System optimization**
  - Larger blocks improve data locality and reduce training time, but lower mini-batch diversity can decrease accuracy
  - A default block size of 128 balances efficiency and accuracy, forming eight blocks per mini-batch of 1,024 target nodes
  - GPU caching reduces repeated feature transfers, with default cache sizes of 2 GB for papers and 100 MB for ogbn-products

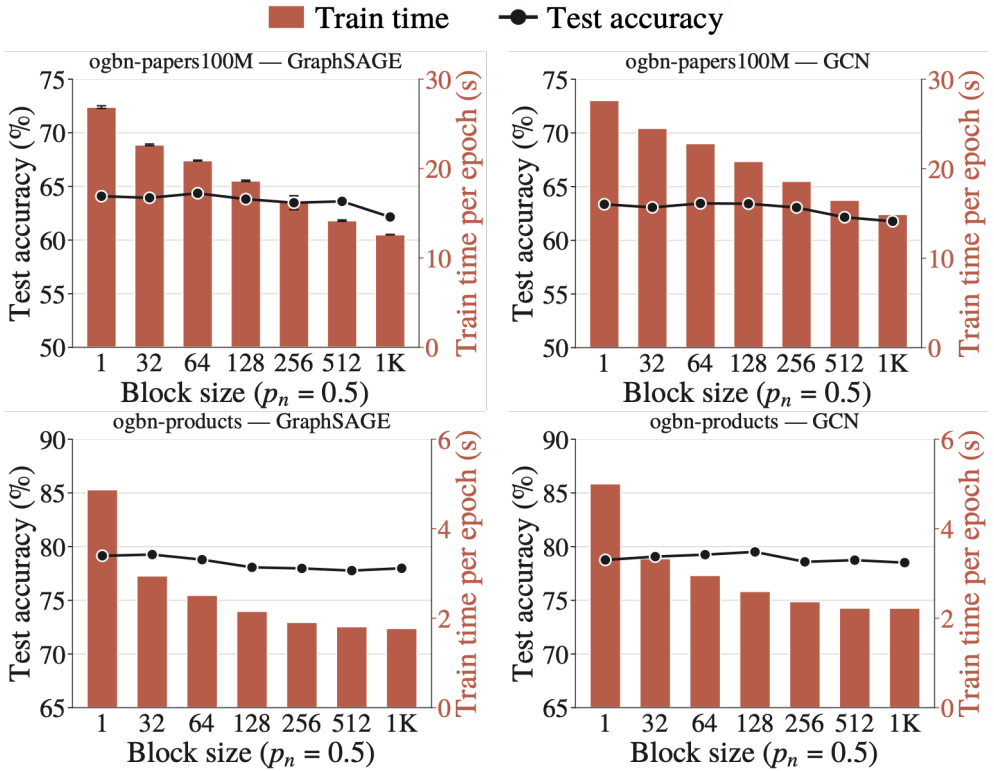


Figure 12 Effect of Block Size on Predictive Accuracy and Training Time

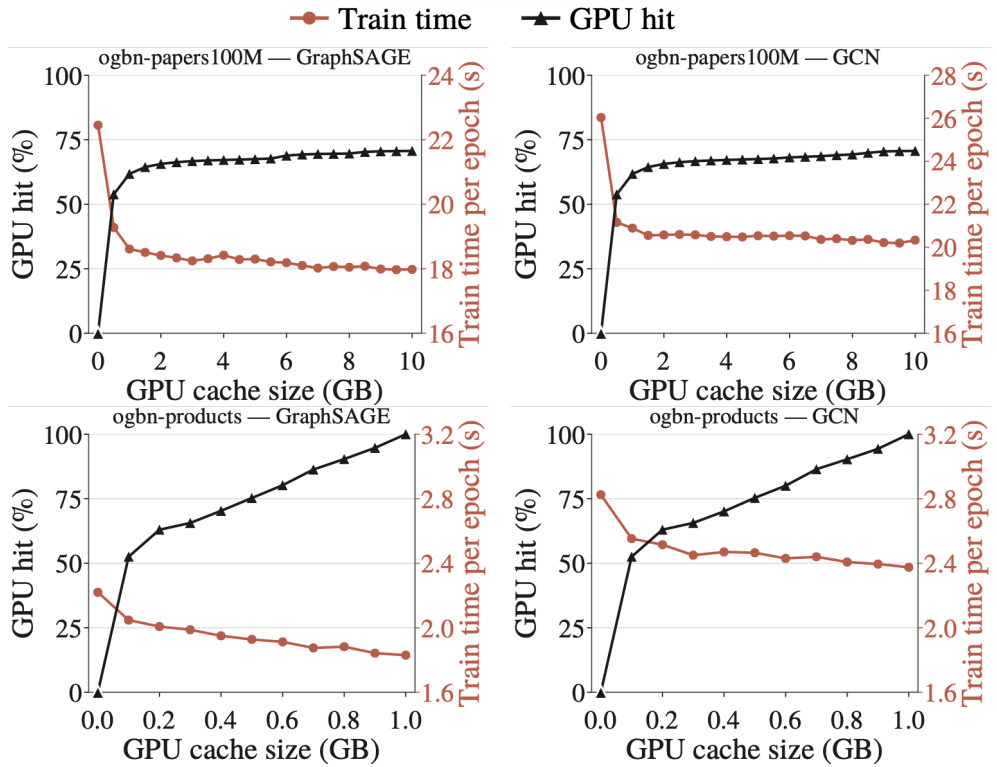


Figure 13 Effect of GPU Cache Size on Hit Rate and Training Time

Conclusions

- **Falcon**
  - Fast and scalable out-of-core GNN training framework that reduces both the graph and neighbor search spaces
  - Improve out-of-core training through community-aware node blocking and GPU caching by enhancing data locality and data transfer efficiency

- **Experiments**
  - Three real-world graphs demonstrate that FALCON outperforms existing methods in most cases
  - Achieving up to 11× speedup over the best-performing competitor while maintaining competitive predictive accuracy

- **Future work**
  - Investigate adaptive sampling strategies that dynamically adjust graph and neighbor sampling according to graph characteristics and training progress.
  - Extend FALCON to larger and more diverse graph workloads and GNN architectures to further examine its scalability and generalizability